

PReFScript:

Partial Recursive Functions for Scripting

Author: José Luis Balcázar, ORCID 0000-0003-4248-4528

Copyleft: MIT License

A Python-based environment to explore and experiment with partial recursive functions; naturally doubles as a (stateless, purely functional) programming language, although it is not intended to be used much as such.

Documentation for version: 2.0.5, **not** backwards-compatible with 1.* **at all**.

Deprecated documentation for previous versions can be found in `doc_v1.md`.

Installation

The usual options should work: `pipx` (less fussy), `pip` inside a virtual environment (recommended), `pip` at global level, which will complain about breaking system packages, or `uv`. Versions 2.0.* offer functionality fully compatible with `pipx`, a feature in which they differ from versions 1.*.

The install command will create a command `prefscript` that you can call from the command line (CLI). It is suggested that your very first call is `prefscript --help`.

Elementary notions

In PReFScript, a script is a sequence of functions, each defined in terms of others and of a few basic functions via the partial recursion rules of composition and minimization.

All functions are from the natural numbers into the natural numbers and may be undefined for some inputs. In order to handle tuples or sequences of natural numbers, a Cantor-like encoding is used.

Cantor-like encoding Before proceeding to our form of partial recursive functions, please see first the companion repository `cantorpairs`. Its README file describes the available functions and their usages. It is a submodule of `prefscript` and provides the related names `dp`, `pr_L`, `pr_R`, `tup_e`, `tup_i`, `s_tup`, `pr`, `seq` as described there. See below for instructions to have them available for practising on your Python REPL.

Basic partial recursive functions The always available basic functions include:

- `k_1`, the constant 1 function;
- `id`, the identity function;

- two functions related to projections of Cantor-encoded sequences: the projection function `proj` and the suffix tuple function `s_tup`;
- addition and multiplication, `add` and `mul` respectively, that interpret the single number received as the Cantor encoding of a pair $\langle x.y \rangle$ and compute the corresponding operation on x and y ; and
- modified difference `diff` that receives likewise a Cantor-encoded pair $\langle x.y \rangle$ and computes $\max(0, x - y)$ so that we always stay within the natural numbers.

Combining functions into new ones Two of the traditional ways of constructing partial recursive functions are composition and mu-minimization (or: linear search, in more modern terminology). In PReFScript, all functions take a single natural number as argument and, if they are defined, return a single natural number. This departs from the original definition where families of different arities were to be defined, with unmanageable cases of indexitis. Here, whenever a function is conceived as taking two arguments (like addition, for one, $\text{add}(x, y) = x + y$) it receives instead a single value that can be interpreted as the encoding of a pair: $\text{add}(z) = x + y$ where $z = \langle x.y \rangle$.

Thus, composition works in the fully standard way: if h is defined by composition of f and g (noted in PReFScript as `comp f g`) then, for all x , $h(x) = f(g(x))$ if $g(x)$ is defined and if f is defined on the resulting value. The minimization operator takes a test function f and creates a new function $h = \mu f$ implementing linear search over f . More precisely, $h(x)$ is computed by testing, in turn, all the values $f(\langle x.0 \rangle)$, $f(\langle x.1 \rangle)$, $f(\langle x.2 \rangle)$, and so on, until finding the smallest value k such that $f(\langle x.k \rangle)$ is nonzero: then $h(x) = k$. In μf one expects f to be a predicate, that is, a total function that only evaluates to 0 or 1. However, nonzero values of f are treated as 1.

A slightly nonstandard addition to the partial recursive function schemes is necessary to handle everything as single natural numbers, namely, given two functions f and g , computing a single value pairing up both outputs: it is expressed as `pair f g` and defines a function h such that $h(x) = \langle f(x).g(x) \rangle$.

Evaluation is eager: an undefined value at any intermediate step leads to the finally desired value remaining undefined. That is, as already stated, if $h(n) = f(g(n))$ and $g(n)$ is undefined then $h(n)$ is undefined, and if one of the intermediate tests $f(\langle x.j \rangle)$ of a minimization turns out to be undefined before reaching the k searched for, then $h(x)$ is undefined.

A form of the so-called *primitive recursion* is also available. Whereas it can be proved that it is redundant in the presence of the given schemes, its lack leads to some computations being inadmissibly slow. We postpone briefly the discussion of this point.

Where did the installation procedure leave example sources?

The installation leaves somewhere in your hard drive half a dozen examples of PReFScript files.

According to a consulted AI, on Windows the **examples** folder containing these files might end up at:

```
%USERPROFILE%\local\pipx\venvs\prefscript\Lib\site-packages\prefscript\
```

On the author's Ubuntu Noble, under a virtual environment named, say, **Xenv**, **pip** installs that folder in

```
~/Xenv/lib64/python3.12/site-packages/prefscript/
```

whereas, outside virtual environments, **pipx** installs the examples folder in

```
~/.local/share/pipx/venvs/prefscript/lib64/python3.12/site-packages/prefscript/
```

Running this may be helpful to locate them:

```
pipx environment | grep PIPX_LOCAL_VENVS
```

For instance, on the author's Ubuntu Noble, that line answers with

```
PIPX_LOCAL_VENVS=/home/balqui/.local/share/pipx/venvs
```

thus providing the most complicated prefix of the path. Users are advised to copy that folder in some other easier-to-remember place. The author harbors hopes that all this will be much easier and more streamlined in future versions of PReFScript.

On a different but related note, if PReFScript was installed via **pipx**, one can find nearby a path like

```
.../python3.12/site-packages/prefscript/cantorpairs/src
```

If the Python REPL is launched from that folder, one can **import cantorpairs** (recommended to import **as cp** for more confort) and run the functions related to the Cantor encoding. Copy the folder **cantorpairs** that you find *below* **cantorpairs/src/** into some easier-to-remember place to be able to use it and experiment with the functions offered.

On the other hand, installations via **pip** into a virtual environment leave **cantorpairs** importable from anywhere provided that the virtual environment is up.

Running the interpreter on a script in a text file

As for the scripts, they contain mainly function definitions.

They may contain as well comments, starting at either the mark **#** or the mark **//** and spanning until the end of the line. They may contain also **#pragma** instructions, handled by an ad-hoc preprocessor and explained below.

Scripts intended to be run *must* include a function definition under the name **main**. In scripts that become imported into other scripts, that name may be missing and, if found, is silently ignored. Running a script consists in calling from the CLI the **prefscript** interpreter with the name of the file containing the script and, optionally, an input integer. The process amounts to calling the **main** function in the script, feeding it the value provided in the CLI call or, alternatively, read from standard input, and writing the outcome to standard output.

Function definitions within these scripts are syntactically very simple: a name for the function followed by a colon, followed by zero or more docstrings in double quotes, finally followed by a specification of how the function is defined. Specifications can be other function names (thus creating aliases, that is, different names of the same function) or the keywords **comp**, **pair**, **mu**, or **rec** (this last one to be clarified below) followed by the adequate number of function specifications: two for **comp** and **pair**, one for **mu** (the test function), and three for **rec**.

Function names follow the standard conventions of alphanumeric characters not starting with a digit, except that names that consist just of digits preceded by three underscores (like `___123`, called “trunders”) are not allowed as they are used internally to provide names to functions that the user code leaves anonymous. Trying to use **import**, **comp**, **pair**, **mu**, or **rec** as a function name is a syntax error.

Parentheses surrounding any function specification are always allowed but never compulsory; users can employ them at will to clarify their code. Parentheses surrounding anything that does not conform syntactically to a function specification are disallowed. Repeated definitions are allowed (e.g. by redefining an imported function) but *only* if they are identical. It is planned for the future to allow them if it can be proved by transitivity that they define the same function, but this feature is not implemented yet.

From a CLI (command line interface) simply call the **prefscript** interpreter followed by the name of the file containing the script, possibly followed by the input. The file extension is assumed to be **.prfs** if nonexistent. CLI flags are available for fine-tuning: **-R**, **--read** changes the criterion by which input is read; **-W**, **--write** changes the criterion by which output is written. Allowed values can be inspected by calling **prefscript --help**. If no input is provided in the call line, reading from standard input is attempted, as with the default **-R**, **--read** flag. Reading from standard input is particularly convenient if you need to call the same script several times on different inputs. Also, it allows a choice of criteria to simplify the way multiple inputs are to be provided.

Also **-I**, **--import_folder** allows one to specify a folder where a file with additional, necessary function definitions can be found. The file itself to be imported is to be specified inside the script itself, by the word **import** followed by a filename in double quotes: it will search for a script of that name, adding the **.prfs** extension if necessary, and will read and have subsequently available

all the function definitions there. It is expected that many scripts will specify `import "std"` which will bring in all the function definitions in the file `std.prfs` provided at installation time in a folder called `stdprfs`:

```
.../python3.12/site-packages/prefscript/stdprfs
```

Accordingly, in order not to “shadow” that file, it is not allowed that a file with name `std.prfs` exists in the user folder or on `import_folder` arguments to the `-I` flag.

These three flags can be also handled from pragmas within the source code (see below).

Additional flags are `-P`, `--show_parsing` that shows the abstract syntax tree of the script and `-G`, `--Goedel_nums` that will provide Gödel numbers of the functions until they skyrocket to over about 300 decimal digits (1000 bits, actually). In both cases, the task is done without running the script.

Recursion

The `rec f g h` construction implements so-called *parameterized course-of-values primitive recursion*. Let’s switch to better names: the function defined by `rec recurse base is_base` receives an integer `z`, tests it to distinguish recursion basis from recursion step, and proceeds accordingly.

For the test and basis, `z` is interpreted as a pair `<param.input>`, where the input part is the actual inductive value and the parameter provides extra information. Then, `is_base` is likely to need to test only `pr_R`, leaving the `param` out (an example follows momentarily).

If the outcome of `is_base` is true (that is, nonzero), `z` is considered to be a basis case and the result is computed as `base(z)`; otherwise, the function `recurse` is applied to a pair consisting of `z` and the whole sequence of values of the function itself that is being defined for all pairs `<param.val>` for `val` between 0 and `input-1`, leaving `param` always invariant. Using these values, `recurse` must obtain the value of the function for `z = <param.input>`.

As an example (somewhat incomplete in that some simple but not basic functions are still missing, such as the constant 0 function `k_0`, the `gt` comparison or the left and right projections) we see how to define the addition via recursion on top of the “add one” function `succ`. The main function is `add_rekurs`, the recursive version of addition, which gets `<x.y>` and must find `x+y` via a recursive construction. Arbitrarily we assign roles: `x` remains as parameter, `y` is taken as inductive variable. To check the base case, `is_zero_R` checks that the right-hand side of the input number is `y == 0`, and then the sum to be computed is `x`, its left-hand side.

The recursive step `add_1_to_prev` gets `<<x.y>.sq>` as input, where `sq` is the whole course-of-values sequence; that is, `<(x+(y-1)).(x+(y-2)). . . .(x+0)>`. This function must take the most recent one and add 1 to it: a call to `pr_R`

selects `sq`, then the composition with `pr_L` fetches its leftmost value, namely `x+y-1`, to which we must add 1.

```
main: add_recurs
```

```
add_recurs:
    rec add_1_to_prev base_case is_zero_R
```

```
is_zero_R:
    comp neg (comp gt pair pr_R k_0)
```

```
base_case:
    "gets called on <x.y> when y is zero hence sum is x"
    pr_L
```

```
add_1_to_prev:
    comp succ comp pr_L pr_R
```

In a case of emergency, `import "std"` allows one to run that script and also to simplify `is_zero_R` into `comp is_zero pr_R` (but make sure to do some exercises and construct on yourself all these utility functions that you will import from `std.prfs` later, once some practice is acquired).

Preprocessor directives

These are used to provide default values to the command line flags `-R`, `--read`, `-W`, `--write`, and `-I`, `--import_folder`. They will be superseded in case the corresponding CLI flags are present. Thus, for each of these three options, there is a default in case neither pragmas nor CLI flags apply (namely, `int`, `int`, and `stdprfs`); if exactly one of them, pragma or flag, is present, it is enforced; and if both are present, the `#pragma` declaration is ignored, being inhibited by the CLI flag. A `#pragma` definition must be placed at the very beginning of a line of its own, with the pragma name immediately followed by a colon and the pragma value being declared.

Call `prefscript --help` to see the allowed pragma values. Here is an example that runs the equality test function on two user-provided integers and writes the answer as a Boolean:

```
#pragma read: intpair
#pragma write: bool
import "std"
main: eq
```

Of course `eq` is one of the functions defined in `std.prfs`. The same effect is obtained if the script only contains the two last lines, without the pragmas, but the call to the interpreter from the CLI includes the flags `--read intpair` `--write bool`.

Character strings

Yet another option to define constant functions is as `asciiconst string` where the string is enclosed in double quotes: it is then handled internally as an integer like all the other functions. Combined with `#pragma read: nothing` and `#pragma write: ascii` (or their equivalent in CLI flags) one can obtain a program printing “Hello, World!” (example `05_hw.prfs`). Admittedly, this is actually the single motivation for the existence of those pragma values and the `asciiconst` construction. In fact, being able to print that string is a suggested condition to be listed some day in webs of esoteric programming languages like esolangs.org.

Directly executable scripts

The mark `#` for introducing comments allows one also to use a “shebang line” in systems having this option: start your `.prfs` file with

```
#!/your/path/to/bin/prefscript
```

(appropriately tuned) and make sure to mark that `.prfs` file as executable. Then you may run it directly, with default flag values. The examples include one such case. If you don’t know what a shebang line is, just ignore this paragraph.